HyperionDev

DOM Manipulation

Task

Visit our website

Introduction

DOM manipulation is a crucial concept in web development, allowing developers to interact with and modify a web page's structure and content in real time. By using JavaScript, we can dynamically update elements, change styles, and respond to user input, making the page more interactive and engaging. This is particularly relevant for creating responsive user interfaces, where content can be updated without needing to reload the page. Mastering DOM manipulation enables developers to create seamless, modern web experiences, which are essential for delivering interactive websites and applications.

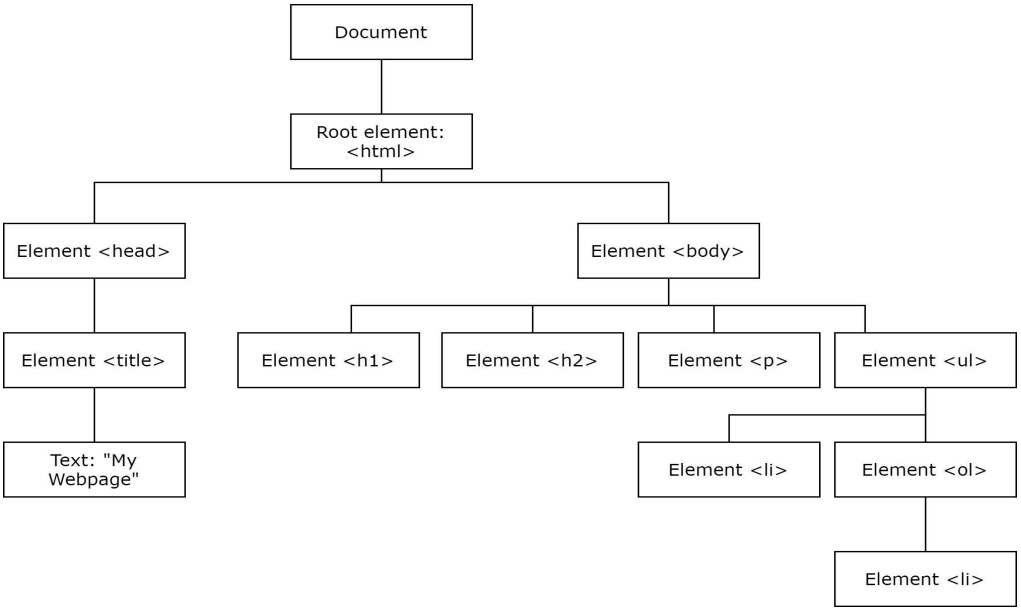
What is DOM manipulation?

Before we begin discussing DOM manipulation, let's clarify exactly what DOM is. DOM stands for Document Object Model and it is the object representation of your HTML file. DOM is essentially a tree of objects where a nested element is branched off from its parent element. Have a look at this skeleton HTML code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My web page</title>
  </head>
  <body>
    <h1></h1>
    <h2></h2>
    <p></p>
    <ul>
      <li></li>
      <ol>
        <li></li>
      </ol>
    </ul>
  </body>
</html>
```

Take note

This would be graphically represented as a tree like this:



We can use this object model to create dynamic pages by manipulating this tree, i.e., the DOM. We can do this by changing, adding, and removing HTML elements like lists, paragraphs, and headings as well as changing CSS style elements. This can all be achieved using JavaScript.

Getting elements

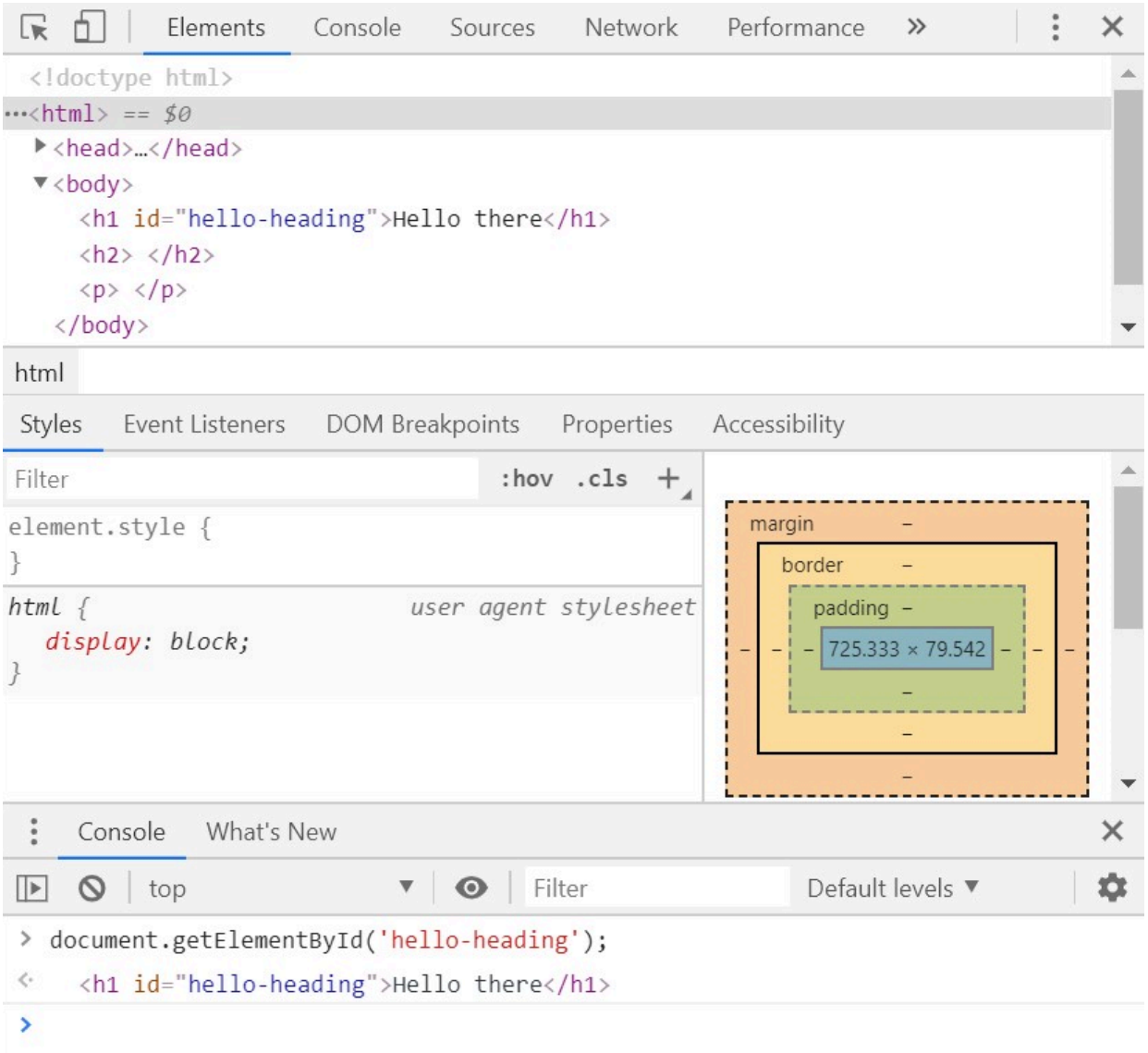
You can use certain functions, such as `document.getElementById("id");`, to get elements by their ID. Additionally, you can get elements using their tags. Let's take a look at the HTML code below:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Webpage</title>
  </head>
  <body>
    <!-- This element will be accessed via its ID for dynamic updates -->
    <h1 id="hello-heading">Hello there</h1>
    <h2></h2>
    <p></p>
  </body>
</html>
```

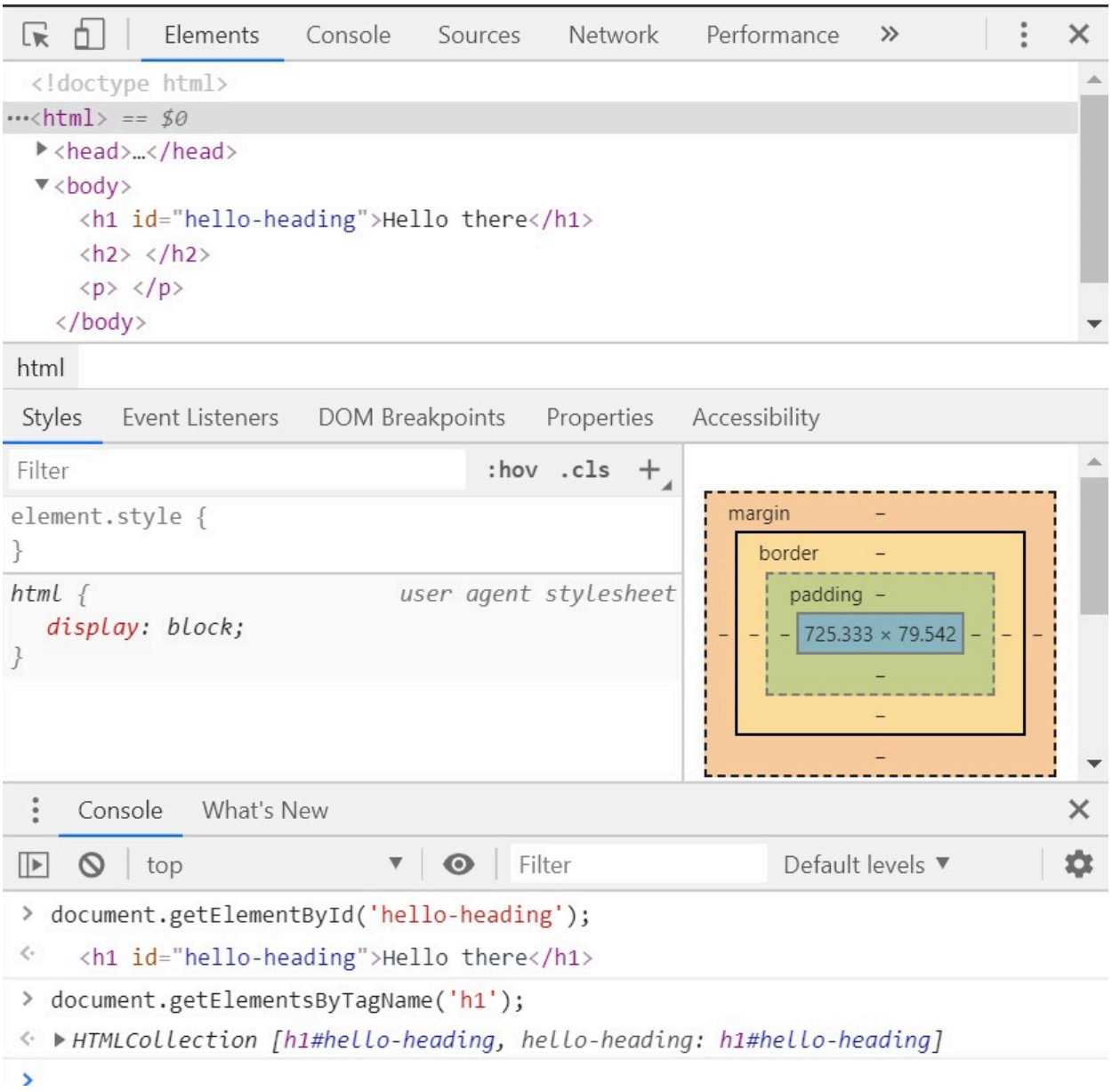
Open this HTML file in Chrome and inspect it:

- **Open Developer Tools:**
    - Firstly, open Developer Tools in your web browser. In Google Chrome, you can do this by **right-clicking** anywhere on the web page and selecting "**Inspect**", or by pressing **F12** or **Ctrl+Shift+I** (**Cmd+Option+I** on Mac). This will open a panel with various tabs for inspecting and debugging your web page.
- **Navigate to the Console tab:**
    - In the developer tools panel, locate and click on the "Console" tab. This is where you can run JavaScript commands and see the output directly.

Great. Now we can retrieve the heading 1 element by its ID. To do this, type `document.getElementById("hello-heading");` into the DevTools console. See the example below:

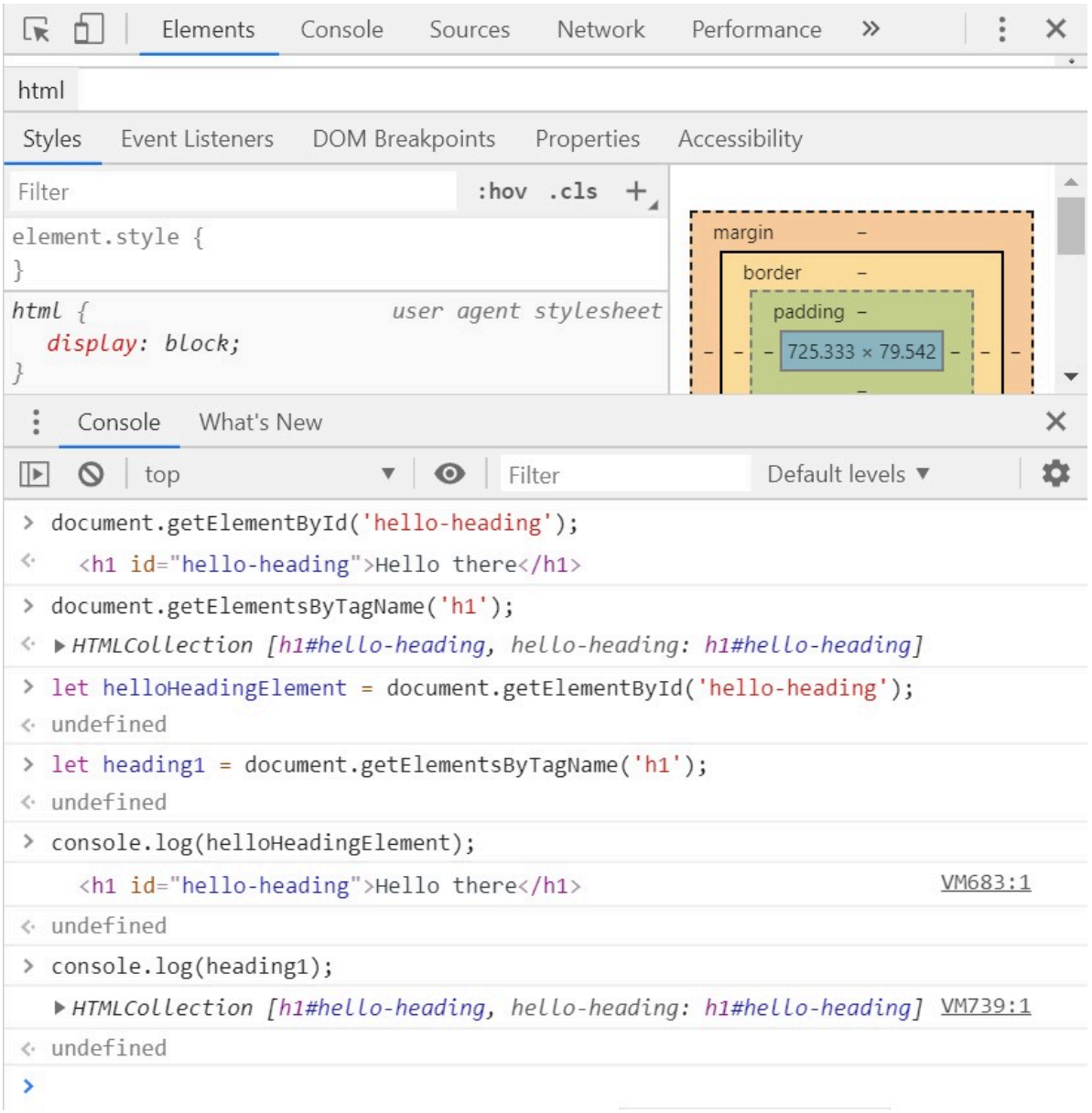


As you can see in the console at the bottom, the `<h1>` element has been returned. We can also retrieve elements by their tag:



This returns an HTML collection of all matching elements. Here, only one `<h1>` element exists, so it is returned. Note the square brackets; while HTML collections resemble arrays, they behave differently.

They auto-update with document changes and do not directly support array methods. To use array methods, you must first convert the collection to an array. These collections can be assigned to variables for easier manipulation. This is done by assigning the `getElement` method's result to a variable. For example:



In the examples above, we assigned the results of both methods to variables and printed them to the console using the Chrome Developer Tools. While these examples were done directly in the console for quick testing, in real projects, you would write this JavaScript code in a separate file and link it to your HTML file. This allows your JavaScript to make the web page more interactive and dynamic.

Note: The "undefined" value appears when executing a let statement because variable declarations do not return a value. Similarly, console.log outputs information to the console but does not return any value, which is why "undefined" is displayed after its execution. This behaviour is normal in JavaScript and does not indicate an error.

Query selector

Using the query selector is another way of getting elements. This is done by returning the first element that matches the CSS selector or ID given. Have a look at the HTML code below:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My web page</title>
  </head>
  <body>
    <h1 id="hello-heading">Hello there</h1>
    <ul id="list-1">
      <li>It's nice</li>
      <li>to meet you.</li>
      <ol id="nested-list-1">
        <li>How</li>
        <li>are</li>
        <li>you</li>
        <li>today?</li>
        <p>Can I offer you some tea or coffee?</p>
        <li></li>
      </ol>
    </ul>
    <p class="big-paragraph">
      We've been having the most lovely weather lately, don't you think?
    </p>
    <br />
    <p class="big-paragraph">What brings you to this part of town?</p>
    <form id="question">
      <input type="text" placeholder="How are you?" />
      <button>Submit</button>
    </form>
    <!-- Link to an external JavaScript file. -->
    <script src="index.js"></script>
  </body>
</html>
```

The JavaScript code examples below demonstrate different ways to use querySelector to select elements. You can add these examples to an external JavaScript file (e.g., **index.js**), which should be linked within your HTML file using a `<script src="index.js"></script>` tag at the end of the body section.

To see the output of the JavaScript code, open your HTML file in a web browser and use the Chrome Developer Tools by right-clicking on the page, selecting "**Inspect**" and then

navigating to the "**Console**" tab. The output will be displayed there, allowing you to see how the JavaScript interacts with the elements on the page.

Return the first element where class="`big-paragraph`":

```
let gettingByClass = document.querySelector(".big-paragraph");
console.log(gettingByClass);
```

Output:

```
<p class='big-paragraph'>We've been having the most lovely weather lately, don't you think?</p>
```

Return the first paragraph element:

```
let firstParagraph = document.querySelector("p");
console.log(firstParagraph);
```

Output:

```
<p>Can I offer you some tea or coffee?</p>
```

Return the first paragraph element where `class="big-paragraph"`:

```
let firstParagraphWithClass = document.querySelector("p.big-paragraph");
console.log(firstParagraphWithClass);
```

Output:

```
<p class='big-paragraph'>We've been having the most lovely weather lately, don't you think?</p>
```

Return an element by ID:

```
let byID = document.querySelector("#nested-list-1");
console.log(byID);
```

Output:

```
<ol id="nested-list-1">
  <li>How</li>
  <li>are</li>
  <li>you</li>
  <li>today?</li>
  <p>Can I offer you some tea or coffee?</p>
  <li></li>
</ol>
```

Return the first list element where the parent is an ordered list:

```
let listOrderedParent = document.querySelector("ol > li");
console.log(listOrderedParent);
```

Output:

```
<li>How</li>
```

Return the specified list element where the parent is an ordered list:

```
let thirdItem = document.querySelector("ol > li:nth-child(3)");
console.log(thirdItem);
```

In the above example, the goal is to return and display a specific child element within an ordered list. The selector "`ol > li:nth-child(3)`" is used to achieve this. Breaking down the selector ol specifies that the search is happening within an ordered list (`<ol>`). `li:nth-child(3)` indicates that we are targeting the third list item (`<li>`) within this ordered list. The > symbol means that we're specifically looking for direct children of the ordered list. Therefore, the third `<li>` element that is a direct child of an `<ol>` is selected and assigned to the variable `thirdItem`.

Output:

```
<li>you</li>
```

To return all instances of a specific element, not just the first one, we use the `querySelectorAll` method. For instance, to select all <p> elements, you write:

```
let paragraphs = document.querySelectorAll("p");
console.log(paragraphs);
```

When you use `querySelectorAll("p")`, you get a **NodeList** of all the `<p>` elements found in the HTML file. For instance, if there are three `<p>` elements in the file, the NodeList will contain these three instances. You can think of a NodeList as a collection of each of the returned elements, such as the `<p>` elements in this case.

Output:

```
NodeList(3) [p, p.big-paragraph, p.big-paragraph]
```

If we want to cycle through a NodeList like the one above, we can use the **forEach** method. The `forEach()` method is an array-like method that allows you to execute a provided function once for each item in the collection. It takes a callback function as an argument, which is called for each element in the NodeList.

```
let paragraphs = document.querySelectorAll("p");
Array.from(paragraphs).forEach(function (paragraph) {
  console.log(paragraph);
});
```

Output:

```
<p>Can I offer you some tea or coffee?</p>
<p class="big-paragraph">We've been having the most lovely weather lately, don't
you think? </p>
<p class="big-paragraph">What brings you to this part of town? </p>
```

Here, we start by casting the items in **paragraphs** to an array (this is not necessary with a NodeList, but is necessary when cycling through an HTML collection). We then use a `forEach()` method that uses a function that then logs each paragraph (i.e., each item) to the console. This can be done to any collection or NodeList that you would like to turn into an array.

Styling elements

We can style a specific element by targeting that element and applying styles dynamically. For example, we have an unordered list (`<ul>`) with the ID `list-1`, and each list item (`<li>`) is a child of this list. To select the second list item, we use the `querySelector` method. The `:nth-child(2)` selector allows us to target the second child element within the parent element, which is the second <li> in the list. The `backgroundColor` and the `style` property allow us to directly modify the inline CSS of the selected element using JavaScript. This means that we can change the appearance of elements on the page without needing to edit an external CSS file.

```
let secondListItem = document.querySelector("#list-1 li:nth-child(2)");
secondListItem.style.backgroundColor = "yellow";
```

If we want to style elements using an external CSS file rather than directly applying styles through JavaScript, we can add a class to the element. To do this, we first select the second list item and use the `classList.add()` method to assign a class to it. This class must be defined in an external CSS file, where we can specify the styles. The `add()` method is used to add the `highlight` class to the second list item.

```
let secondListItem = document.querySelector("#list-1 li:nth-child(2)");
secondListItem.classList.add("highlight");
```

In the external CSS file, you would define the `.highlight` class like this:

```
.highlight {
  background-color: yellow;
}
```

Note: To ensure the styles are applied, do not forget to link the stylesheet in your HTML file by placing the following line within the `<head>` element:

```
<link rel="stylesheet" href="styles.css" />
```

Appending to elements

We can add both text and new elements to existing elements in our HTML file. If we want to see the text of a current element, we can use `.textContent`. For example:

```
let firstParagraphText = document.querySelector("p").textContent;
console.log(firstParagraphText);
```

If we wanted the console to log all paragraph elements that we saved in the paragraphs variable above, we can use a `forEach` loop:

```
let paragraphs = document.querySelectorAll("p");
Array.from(paragraphs).forEach(function (paragraph) {
  console.log(paragraph.textContent);
});
```

We could also append to any current text as we would with any other string. For example:

```
let firstParagraphText = document.querySelector("p");
firstParagraphText.textContent += "!";
console.log(firstParagraphText);
```

This will add an exclamation mark to the text. If, however, we wanted to change the text completely, we could simply reassign the `textContent` property of the element:

```
let firstParagraphText = document.querySelector("p");
firstParagraphText.textContent = "I'm a brand new paragraph";
console.log(firstParagraphText);
```

We can also change an element's content to include different HTML elements and text using the `innerHTML` property. This property allows you to add or replace the HTML content inside an element.

```
let firstParagraphText = document.querySelector('p');
firstParagraphText.innerHTML += "<h2>Good day</h2>";
```

In the above example, the `innerHTML` property is used to add an `<h2>` element with the text "Good day" inside the first `<p>` element. Using `+=`, the new HTML can be appended without removing the existing content.

Creating new elements

While we can modify existing elements, it is also possible to create entirely new elements using the `createElement` method in JavaScript. This allows us to add new elements to the DOM. For instance, if we want to add a new list item `<li>` to the existing unordered list `<ul>`, we can do the following:

```
let list1 = document.querySelector("#list-1");
let listItem = document.createElement("li");
listItem.textContent = "Hello again";
list1.appendChild(listItem);
```

In the above example, we start by selecting the unordered list `<ul>` with the ID `list-1` using `document.querySelector("#list-1");`. This allows us to work with that specific list in our code.

Next, a new list item `<li>` is created and stored in a variable called `listItem`. We set the text inside this new list item to "`Hello again`" using `listItem.textContent`.

Finally, the new list item is added to the end of the selected list using `list1.appendChild(listItem)`. The newly created and added list element is now part of the list and will be displayed as the last item in the list.

Understanding DOM manipulation opens up a world of possibilities for creating interactive and dynamic web experiences. As you continue exploring JavaScript, these skills will be key for building modern websites and applications that respond to user actions without reloading pages. This is just the beginning—mastering DOM manipulation will allow you to craft more seamless and engaging user interfaces in the future.

Instructions

Open the **example** file in Visual Studio Code and read through the comments before attempting these tasks.

Practical task 1

Follow these steps:

- In this task, you will be required to create a shopping list by manipulating the HTML DOM.
- Do so by following these steps:
    1. Open the template file **index.html**.
    2. Create a JavaScript file called **main.js** and link it in **index.html** using a `<script>` tag.
    3. In **main.js**, create an array and initialise it with at least four grocery items.
    4. Create a function called **displayItems**, which will display each item in the array as list elements in the `<ul>` tag. You will need to use the `<ul>` tag's ID.
    5. Create a function called **setDefaultChecked** that changes the CSS styling of two list items to indicate they have been bought. The purpose of the **setDefaultChecked** function is to mark certain items as checked or completed, providing a visual cue to the user that these items have already been bought.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
